

s4.01 — Healthcare ingestion pipeline for millions of records

Index

- [Interview framing](#)
- [Requirements — IC3 baseline](#)
- [Capacity and consequence — IC3 baseline](#)
- [Architecture and flow — IC3 baseline](#)
- [API and data model — IC3 baseline](#)
- [Record identity, acceptance boundary, and idempotent apply — IC3 baseline](#)
- [Quarantine, replay, and reconciliation — IC3 baseline](#)
- [Failure, operations, and security](#)
- [Alternatives and IC4 stretch](#)
- [Level signal — IC3 baseline vs IC4 stretch](#)
- [Interview pushback and validation](#)
- [Recall card](#)
- [References](#)

Interview framing

- Target: OCI IC3 ownership of a bounded platform component.
 - `IC3 baseline` is what must land in every phase.
 - `IC4 stretch` adds architecture changes under stronger constraints.
 - These are preparation labels, not Oracle's official rubric.
- Evidence: first-hand `1p3a` prompt retained in [the bank](#).
 - Prompt as reported: design an ingestion pipeline for millions of healthcare records.
 - The reported discussion covered ingestion, broker choice, storage, processing, fault tolerance, scalability, monitoring, and failed-record handling.
 - This entry separates the healthcare variant previously embedded in the metrics answer.
 - The architecture below is a proposed interview answer.
 - It does not claim to reproduce Oracle's implementation, an interviewer's accepted answer, or FHIR endpoint semantics.
- Why this is hard
 - Naive: an HTTP endpoint that writes each record into a database and returns `200`.
 - First break: a source retries a timed-out batch and the same lab result appears twice
 - A duplicate observation in a patient chart is a clinical error, not a metrics blip.
 - Second break: one malformed record in a `10,000`-record batch, and the pipeline either drops the whole batch or silently skips the bad record
 - Either answer loses data that a regulator will ask about later.
 - Third break: a correction arrives before the original, or a replay reprocesses last week's batch with new record identities
 - "Current" values go backward or double.
 - Minimum primitives
 - A stable source record identity plus a source version on every record
 - One explicit acknowledgment boundary that the source can rely on
 - An immutable raw tier that outlives every downstream projection
 - An idempotent apply rule keyed by identity and version
 - A quarantine with the same identity discipline as the happy path

- 🗨️ “Metrics can drop a sample; a healthcare record can only be accepted, versioned, or quarantined, never lost, never doubled.”
- Opening script
 - “I will treat this as a durable, replayable record-ingestion pipeline where acceptance is an explicit contract, not a transport acknowledgment.”
 - “First I confirm the record shape, the per-patient ordering requirement, the retention horizon, and what a source expects when we say ‘accepted’.”
 - “Then I size the daily volume and the raw tier, which tells me the broker is small and the storage and reconciliation problems are the real ones.”
 - “The architecture is a control plane for sources, runs, and replays, and a data plane of an append log, validators, an immutable raw tier, and idempotent projections.”
 - “The deep dive is record identity and versioning at the acceptance boundary, and how quarantine and replay reuse that identity so counts reconcile.”
 - “I will close with failure recovery at each acknowledgment boundary, privacy in logs and errors, and where the design changes for per-patient strict ordering at ten times the load.”

Phase	Clock	Deliverable
Intro and behavioral slice	00:00–05:00	90-second pitch; confirm the prompt
Requirements	05:00–11:00	Scope, clarifying answers, primary invariant
Estimates and contract	11:00–17:00	Daily volume, raw tier size, partition count; API semantics
Architecture	17:00–30:00	Diagram, control/data plane split, one batch trace
Deep dive	30:00–45:00	Identity and version, acknowledgment boundary, quarantine and replay
Operations and tradeoffs	45:00–50:00	Failure table, reconciliation, privacy, wrap script
Wrap-up	50:00–54:00	Four-sentence recap and weakest assumption
Questions for interviewer	54:00–60:00	Two role-specific questions

- Clock note
 - If the interviewer states 45 minutes, drop the intro to 2 minutes and compress each phase proportionally.
 - The interviewer's clock wins over this table.

Requirements — IC3 baseline

- Numerical targets are interview assumptions unless an adjacent primary source states otherwise.

Question	Assumed answer	Why it changes the design
What is a record and how does it arrive?	Structured clinical events (observations, encounters, orders), 1–10 KB each, in batches from hospital or lab systems	Batch-level acceptance plus record-level results; payloads stay off the metadata database
How many sources and tenants?	Hundreds of facilities across tens of tenants; one tenant per health system	Tenant and facility are part of identity, authorization, and partitioning
Is per-patient ordering required?	Yes within a facility; no global order	Partition key includes patient; ordering is per key, never cross-facility
Do records get corrected or deleted?	Yes, corrections are common; deletion requests exist	Every record carries a source version; history is append-only with explicit “current” rules
What does “accepted” mean to the source?	Durably persisted and replayable; not yet queryable	Acknowledge after the durable append boundary, expose a run state for application
Retention horizon?	Raw records retained for years under policy; assumed 7 years	An immutable raw tier in object storage, not the broker, is the system of record
Freshness target?	Queryable within minutes, not seconds	Asynchronous apply with a freshness SLO; no synchronous fan-in on the accept path

- Functional scope

- Register sources with schema references and credentials.
- Accept record batches by object reference or bounded inline payload.
- Persist raw batches immutably before any acknowledgment.
- Validate, normalize, and apply records to a versioned clinical store.
- Quarantine invalid records with diagnostics and an authorized repair or replay path.
- Expose run state, per-record results, and reconciliation counts.
- Serve authorized subject-scoped queries over normalized records.

Requirement	Target	Enforced by
Accepted-record durability	Zero loss after 202; survives loss of one storage node and one process	Acknowledge only after raw object commit and replicated log append
Duplicate suppression	0 duplicate domain effects per (source, record, version, transform)	Unique key on the run and record result tables; idempotent upsert on the clinical store
Freshness	p95 accepted-to-queryable under 5 min; p99 under 15 min	Consumer lag alarm at 2 min; apply workers scale on lag
Accept latency	p99 under 500 ms for a batch by reference; inline batch capped at 1,000 records	Reference validation only; no synchronous domain apply
Availability of acceptance	99.9% monthly	Stateless accept fleet; replicated log; raw tier in regional object storage
Replay window	7 days of log retention; any age from the raw tier	Broker retention policy; raw objects are immutable and never expire before policy
Quarantine visibility	Every failed record has an error code within 15 min of acceptance	Validator writes a result row for every record, success or failure
Reconciliation	Source-reported count equals accepted, and accepted equals applied plus quarantined, per run	Run counters advance only from result rows; alarm on mismatch older than 30 min
Privacy	No patient identifiers in logs, metrics labels, or error messages	Redaction at the validator; error rows store codes and field paths only

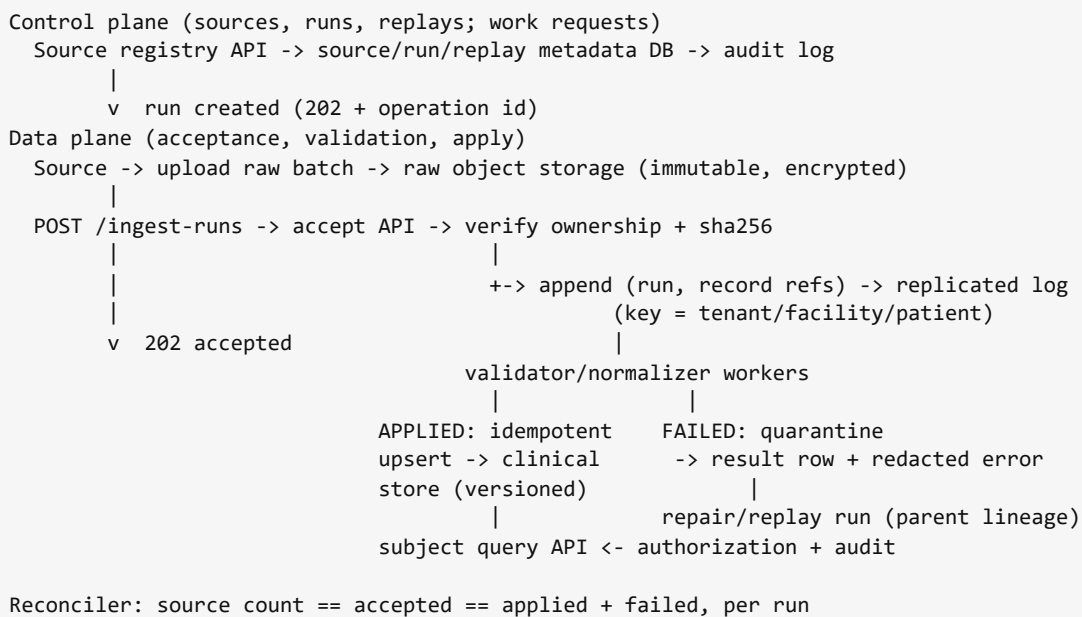
- Invariants
 - A record is identified by (tenant, source, source_record_id, source_version).
 - A retry reuses identity, so a retried batch cannot add a second domain effect.
 - A correction creates a newer version, never an in-place overwrite of history.
 - Every accepted record ends in exactly one terminal result: APPLIED or FAILED.
 - The raw tier is immutable and is the only system of record; every projection is rebuildable from it.
 - Authorization is scoped by tenant and permitted facility or patient context on every read and replay.
- Non-goals
 - FHIR-compliant endpoint semantics or terminology mapping correctness.
 - Real-time clinical alerting on ingested values.
 - Cross-tenant patient matching or deduplication.
 - Analytics warehouse design beyond the normalized store.

Capacity and consequence — IC3 baseline

- Volume
 - Assume 50 million records per day, average 4 KB.
 - Average rate is about 580 records/s; a 5x peak is about 3,000 records/s.
 - Peak byte rate is about 12 MB/s, which one broker partition set handles easily.
 - Decision: the broker is not the bottleneck
 - Spend design time on identity, storage, and reconciliation rather than throughput.
- Raw tier
 - 50 M × 4 KB = 200 GB/day of raw payload.

- At 7 years retention that is about 510 TB uncompressed, roughly 100–170 TB at 3–5× compression.
- Decision: raw records live in object storage, not in the broker or the SQL store
 - The broker keeps 7 days for replay; the raw tier keeps the policy horizon.
- Partitions
 - Per-patient ordering requires one key per (tenant, facility, patient_token).
 - 3,000 records/s over 64 partitions is about 50 records/s per partition, far below a single consumer's capacity.
 - Decision: over-partition at 64 per region and key by patient
 - A skewed facility that owns 30% of the volume still fits in a few partitions; alarm on per-partition lag.
- Metadata rows
 - Batches of 10,000 records give about 5,000 runs per day.
 - One result row per record is 50 M rows per day, roughly 100 B each, so about 5 GB/day of result rows.
 - Decision: partition result rows by (tenant, run) and time
 - Retain results for 90 days hot; older results are rebuildable from the raw tier.
- Replay catch-up
 - A 24 h outage leaves 50 M records to apply.
 - At 4× the average apply capacity, catch-up takes about 6 h; at 2× it takes 24 h.
 - Decision: reserve apply capacity for live traffic and cap replay workers at a fixed share
 - Replay must not starve live ingestion; catch-up time is a published number, not a hope.

Architecture and flow — IC3 baseline



- Control plane owns sources, runs, replays, and authorization metadata.
 - Mechanism bought: every batch is a work request with an operation id, so state is queryable and retries are safe.
 - Cost: a control-plane outage blocks new runs and replays; the design must say what still works.
- Accept API owns the acknowledgment boundary.
 - Mechanism bought: 202 means "raw object committed and log append replicated", nothing more and nothing less.
 - Cost: the source must keep a retryable copy until 202, and the API must tolerate the same batch twice.
- Raw object storage owns the system of record.
 - Mechanism bought: immutable, encrypted, policy-retained payloads; every projection is rebuildable.
 - Cost: two durable writes per batch (object plus log) and a reconciliation between them.

- Replicated log owns ordering and replay.
 - Mechanism bought: per-key order, consumer offsets, and a bounded replay window.
 - Cost: retention is finite, so long-horizon replay must go through the raw tier.
 - Validators and apply workers own transformation and idempotent application.
 - Mechanism bought: schema and version checks, redaction, and an upsert keyed by identity and version.
 - Cost: application is asynchronous, so freshness needs its own SLO and lag alarm.
 - Clinical store owns the versioned normalized projection.
 - Mechanism bought: subject-indexed reads with full version history and lineage.
 - Cost: "current" is a derived rule that must be stated, not last-arrival-wins.
 - Reconciler owns proof of completeness.
 - Mechanism bought: per-run counts and identity-level diffs that explain every gap.
 - Cost: extra reads and a second source of truth that must itself be idempotent.
 - Static stability
 - Registered sources with valid credentials can continue submitting batches while the control-plane metadata DB is read-only or degraded, because acceptance needs only the source record and the log.
 - New source registration, replays, and error listing wait for the control plane.
 - Shared mechanisms
 - Durable ingestion, consumer offsets, and backpressure follow the [metrics pipeline](#).
 - This design replaces the metrics pipeline's lossy policies with an explicit business-record acceptance contract.
- Source uploads the batch object to tenant-scoped storage and computes its `sha256`.
- Source calls `POST /ingest-runs` with `sourceBatchId`, `schemaVersion`, `objectRef`, and the digest.
- Accept API verifies object ownership, size, and digest, then creates the run under the unique `(source, batch, transform)` key.
- Accept API appends one log entry per record reference, keyed by `(tenant, facility, patient_token)`.
- Log replication reaches the required in-sync replicas and the API returns `202 {runId, operationId}`.
- Validators consume in key order, check schema and source version, and redact identifiers.
- Apply workers upsert `(source_record_id, source_version)` into the clinical store and write an `APPLIED` result row.
- Invalid records receive a `FAILED` result row with an error code and field path; payloads stay in the raw tier.
- Reconciler compares source-reported, accepted, applied, and failed counts and marks the run `COMPLETED` or `NEEDS_ATTENTION`.

API and data model — IC3 baseline

- The detailed contract lives in the [healthcare companion](#); this table restates it with idempotency and conflict behavior.

Surface	Contract	Conflict/error behavior
<code>POST /ingest-sources</code>	<code>{name, schemaRef, credentialRef}</code> ; returns <code>201 source</code>	Same name in tenant returns <code>409</code> ; administrator role only
<code>POST /ingest-runs</code>	<code>{sourceId, sourceBatchId, schemaVersion, objectRef, sha256}</code> ; returns <code>202 {runId, operationId}</code>	Same <code>(source, batch, transform)</code> returns the prior run; different digest for the same batch id returns <code>409</code>
<code>POST /ingest-sources/{id}/record-batches</code>	Bounded inline <code>{batchId, schemaVersion, records}</code> , at most <code>1,000</code> records; returns <code>202 run</code>	Same batch id with different content returns <code>409</code> ; over-size returns <code>413</code>
<code>GET /ingest-runs/{id}</code>	<code>{state, received, validated, applied, failed, watermark}</code>	Counters distinguish acceptance from application; <code>404</code> across tenants
<code>GET /ingest-runs/{id}/errors?after=</code>	Cursor page of redacted per-record errors	Restricted role; no payload values in the response

Surface	Contract	Conflict/error behavior
POST /ingest-runs/{id}/replays	{recordIds?, transformVersion, reason}; returns 202 replay run	Same (parent run, record set, transform) returns the prior replay; missing reason returns 400
GET /clinical-records?subjectId=&after=	Normalized record page under subject and time filters	Fine-grained authorization plus audit; no bulk export for ordinary readers

State	Key	Fields/invariant
Source	(tenant_id, source_id)	Name, schema reference, credential reference; unique name per tenant
Run	(tenant_id, run_id)	Source, source_batch_id, transform_version, schema_version, raw object ref, payload hash, parent run, state; unique (tenant, source, batch, transform)
Record result	(tenant_id, run_id, record_id)	State PENDING, APPLIED, or FAILED; error code; output ref; partial index on FAILED
Clinical record	(tenant_id, source_id, source_record_id, source_version, transform_version)	Subject token, observed time, encrypted payload ref, lineage run id; append-only
Subject index	(tenant_id, subject_token, observed_at, source_record_id)	Serves subject-scoped reads without scanning versions
Log offset	(consumer_group, partition)	Committed only after the result row for every record at or before that offset exists

- Reference DDL
 - Keys and uniqueness match the state table above and the [companion schema](#).
 - Raw and encrypted payloads live in object storage and the consumer offset lives in the log's consumer-group store; only sources, runs, results, and normalized record metadata are relational.

```

CREATE TABLE ingest_sources (
  tenant_id text NOT NULL, source_id text NOT NULL, name text NOT NULL,
  schema_ref text NOT NULL, credential_ref text NOT NULL,          -- vault handle, never the credential
  PRIMARY KEY (tenant_id, source_id),
  UNIQUE (tenant_id, name)
);
CREATE TABLE ingest_runs (
  tenant_id text NOT NULL, run_id text NOT NULL, source_id text NOT NULL,
  source_batch_id text NOT NULL, transform_version text NOT NULL,
  schema_version text NOT NULL, raw_object_ref text NOT NULL,
  payload_hash text NOT NULL,                                     -- sha256; same batch id with another hash
  is 409
  parent_run_id text,                                           -- set on replay runs
  state text NOT NULL, accepted_at timestampz NOT NULL,
  PRIMARY KEY (tenant_id, run_id),
  UNIQUE (tenant_id, source_id, source_batch_id, transform_version),
  FOREIGN KEY (tenant_id, source_id) REFERENCES ingest_sources,
  FOREIGN KEY (tenant_id, parent_run_id) REFERENCES ingest_runs,
  CHECK (state IN ('ACCEPTED', 'APPLYING', 'COMPLETED', 'NEEDS_ATTENTION'))
);
CREATE TABLE ingest_record_results (
  tenant_id text NOT NULL, run_id text NOT NULL, record_id text NOT NULL,
  state text NOT NULL, error_code text,                          -- code only; no payload values
  output_ref text,
  PRIMARY KEY (tenant_id, run_id, record_id),
  FOREIGN KEY (tenant_id, run_id) REFERENCES ingest_runs,
  CHECK (state IN ('PENDING', 'APPLIED', 'FAILED')),
  CHECK (state <> 'FAILED' OR error_code IS NOT NULL)
);
CREATE INDEX ingest_failed_records ON ingest_record_results (tenant_id, run_id, record_id)
  WHERE state = 'FAILED';                                         -- serves the errors cursor page
CREATE TABLE clinical_records (
  tenant_id text NOT NULL, source_id text NOT NULL, source_record_id text NOT NULL,
  source_version bigint NOT NULL, transform_version text NOT NULL,
  subject_token text NOT NULL,                                   -- from the tokenization service
  observed_at timestampz NOT NULL,
  encrypted_payload_ref text NOT NULL,                           -- object-storage handle
  lineage_run_id text NOT NULL,
  PRIMARY KEY (tenant_id, source_id, source_record_id, source_version, transform_version),
  FOREIGN KEY (tenant_id, source_id) REFERENCES ingest_sources,
  FOREIGN KEY (tenant_id, lineage_run_id) REFERENCES ingest_runs
);
CREATE INDEX clinical_subject ON clinical_records
  (tenant_id, subject_token, observed_at, source_record_id);     -- the subject index row of the state table

```

- What the DDL does not capture
 - The `Subject` index row is the secondary index `clinical_subject`, not a table; the `Log offset` row is the log's consumer-group offset, committed after the result-row transaction, so a crash between them replays and the insert-if-absent on `clinical_records` absorbs the duplicate.
 - `received`, `validated`, `applied`, and `failed` are counted from `ingest_record_results` at read time, never stored as counters.
 - Deviation from the companion: the state enums, the `parent_run_id` self-reference, and the lineage foreign keys are added here; table and column names are unchanged.
- Conventions
 - Run states: `ACCEPTED` → `APPLYING` → `COMPLETED` or `NEEDS_ATTENTION`.
 - `received`, `validated`, `applied`, and `failed` advance only from result rows, never from in-memory counters.
 - Subject tokens come from a trusted tokenization service; an unsalted hash of an identifier is not sufficient privacy protection.
 - Raw and encrypted payloads live in object storage; SQL stores bounded metadata and redacted errors.
 - Raw records, errors, and normalized history carry independent policy-driven retention.
 - Shared conventions follow the [common API rules](#).

Record identity, acceptance boundary, and idempotent apply — IC3 baseline

- Identity
 - Every record carries (tenant, source, source_record_id, source_version) assigned by the source.
 - Tenant and facility are part of identity and of authorization, so the same record id from two facilities never collides.
 - A retry reuses the identity, and a correction increments source_version.
 - The pipeline never mints a replacement identity for a record it already holds.
- Three timestamps
 - Clinical event time, source sequence or version, and ingestion time are different values.
 - Preserve all three; never choose last-arrival-wins as the ordering rule.
 - "Current" for a subject is the highest source_version for each source_record_id, evaluated at read time.
- Acceptance boundary
 - 202 is returned only after two durable writes: the raw object commit and the replicated log append.
 - Before that boundary the source retains a retryable copy and the API is free to fail.
 - Buffers are finite; when the log or raw tier is unavailable the API returns 503 with a retry hint rather than promising an unbounded spool.
 - A 202 never claims that records are queryable; run state does that.
- Duplicate batch
 - The run table has a unique key on (tenant, source, source_batch_id, transform_version).
 - A retry with the same batch id and digest returns the existing run and operation id.
 - A retry with the same batch id and a different digest returns 409, because silently accepting it would hide a source-side bug.
- Crash between the two durable writes
 - Raw object committed, log append lost: the run row does not exist, the source retries, and the object is overwritten with identical bytes or reused by digest.
 - Log appended, run row lost: the append carries the run id, so run creation is retried from the log entry by a repair worker before consumers process it.
 - Both committed, 202 lost: the source retries and receives the same run from the unique key.
- Ordering
 - Partition by (tenant, facility, patient_token) so that one patient's records arrive in order at one consumer.
 - Source sequence or version handles late and duplicate updates within that key.
 - No global ordering across facilities is claimed or needed.
- Idempotent apply

```
APPLY(record):
    key = (tenant, source, source_record_id, source_version, transform_version)
    if result_row(run, record_id).state in {APPLIED, FAILED}:
        return # already terminal for this run
    validate schema_version and required fields; redact identifiers
    if invalid:
        write result_row FAILED with error_code and field_path # no payload
        return
    upsert clinical_record(key) with payload_ref, lineage_run_id
    # insert-if-absent; an existing row with the same key is a no-op
    write result_row APPLIED in the same transaction as the upsert
    commit
consumer offset advances only after every record at or before it has a result row
```

- Consistency boundary
 - The clinical upsert and the result row commit in one transaction, so a crash cannot apply a record without recording it or record it without applying it.

- The consumer offset is committed after that transaction, so a crash replays the record and the insert-if-absent makes the replay a no-op.
- The raw object and the log are outside that transaction; the run-level unique key and the repair worker reconcile them.
- Exact failure this prevents
 - A source retries a 10,000-record batch after a timeout.
 - Without the unique run key and versioned identity, 10,000 observations appear twice in patient charts.
 - With them, the retry returns the same run and the apply step finds every key already present.
- Schema versions
 - Producers upgrade at different times, so the validator accepts every supported `schema_version` and rejects unknown ones with a code.
 - Transform code is versioned separately, so a replay can state which transformation produced which row.

Quarantine, replay, and reconciliation — IC3 baseline

- Quarantine
 - A failed record keeps its source identity, run id, error code, and field path.
 - The payload stays in the raw tier; the result row never contains payload values.
 - Permanent invalidity (schema violation, missing required field) and temporary downstream outage (clinical store unavailable) are distinct states.
 - Temporary failures retry with backoff inside the consumer; permanent failures go to `FAILED` immediately.
- Partial batch
 - Independent valid records continue when the domain permits partial application.
 - When a batch is declared atomic by the source, the run stays in `NEEDS_ATTENTION` until every record is applied or the batch is withdrawn.
 - Which mode applies is a source-level setting, not a per-request guess.
- Repair and replay
 - `POST /ingest-runs/{id}/replays` creates a child run with `parent_run_id` and the original record identities.
 - A replay may select specific `recordIds` or the whole run, and names the `transformVersion` it will use.
 - The apply rule is unchanged, so a replay with the same source version and transform version cannot add duplicate domain effects.
 - A replay with a new transform version appends new rows under that version and leaves prior rows for lineage.
 - Every replay carries an actor, a reason, and an audit record.
- Replay resource bound
 - Replay consumers run in their own consumer group with a fixed share of apply capacity, assumed 25%.
 - Live ingestion keeps its freshness SLO during a large replay.
- Reconciliation
 - Per run: source-reported count equals accepted count, and accepted equals applied plus failed.
 - Per identity: a diff of expected record identities against result rows explains every gap.
 - A duplicate and a missing record can cancel out in a total count, so matching aggregates are not proof of correctness.
 - Runs whose counts do not settle within 30 min move to `NEEDS_ATTENTION` and alarm.
- Deletion and correction requests
 - A deletion request is tracked through raw, normalized, indexed, and archived copies using lineage.
 - The raw tier applies deletion under policy, not on demand, because it is the record of what was received.
 - Reads of a deleted record return nothing; result rows keep the identity so reconciliation still explains the gap.
- Privacy and access
 - Transport and storage are encrypted; payload objects are encrypted under tenant-scoped keys.

- Reads are scoped by tenant and permitted facility or patient context.
- Access and replay operations are audited with actor, reason, and affected identities.
- Sensitive payloads never appear in routine error logs, metrics labels, or traces.

Failure, operations, and security

Failure	Blast radius	Detection	Degraded behavior	Recovery
Accept response lost after 202	One batch	Source timeout	Source retries the same batch id	Unique run key returns the prior run
Raw object committed, log append failed	One batch	No run row; source retry	503 with retry hint	Retry reuses the object by digest
Log appended, run row missing	One batch	Repair worker finds orphan run id	Consumers hold the partition until the row exists	Repair worker recreates the run row from the log entry
Validator crash mid-batch	One partition's in-flight records	Consumer group rebalance	Records replay from the last committed offset	Insert-if-absent makes replayed applies no-ops
Clinical store unavailable	All apply workers	Apply error rate, lag alarm	Acceptance continues; freshness SLO degrades	Consumers resume from offsets; lag drains at the reserved capacity
Skewed facility floods one partition	One partition's patients	Per-partition lag	Other partitions unaffected	Rebalance keys by facility sub-hash; source-level rate limit
Malformed batch or wrong digest	One batch	Accept-time verification	409 or 400; nothing appended	Source resubmits with a new batch id
Replay overload	Live freshness	Apply lag on the live group	Replay group capped at its share	Pause replay group; resume after live lag clears
Control-plane metadata store unavailable	New sources, replays, error listing	Health check, error rate	Registered sources keep submitting; consumers keep applying	Fail over replica; queued run rows reconcile from the log
Regional object storage unavailable	All acceptance in the region	Object write failures	503; sources hold retryable copies	Restore storage; no acknowledged record is lost

- Observe
 - Accepted-to-applied lag per partition and per tenant.
 - Oldest run in APPLYING and oldest run in NEEDS_ATTENTION.
 - Failed-record rate by error code and by source.
 - Reconciliation mismatches per run.
 - Replay share of apply capacity.
 - Redaction failures, which are a security signal.
- Retry discipline
 - Sources retry acceptance with exponential backoff and jitter, reusing the batch id.
 - Consumers retry temporary apply failures with bounded attempts before parking the partition.
 - A schema violation or authorization failure is never retried; it produces a FAILED row or a 4xx.
 - Replay is an explicit operator action with a reason, not an automatic retry.
- Tenant isolation
 - Raw objects, encryption keys, log keys, and result partitions are scoped per tenant.
 - Per-tenant quotas cap batches per hour and records in flight so one health system cannot starve another.
 - A tenant's token receives 404 on another tenant's run ids, not 403, to avoid existence leaks.
 - Facility-level authorization applies within a tenant for reads and replays.
- Rollout and rollback safety
 - Schema and transform versions roll forward under expand, migrate, contract; old producers stay accepted during the mix.

- A new validator version must accept every schema version the previous one accepted.
- Canary apply workers on one partition set and compare result-row distributions before fleet rollout.
- Rollback keeps rows written under the newer transform version; lineage marks them and a replay can supersede them.
- Acceptance survives a deploy because the log and raw tier do not depend on validator availability.

Alternatives and IC4 stretch

Option	What it is	Strength	Weakness	Choose when
Synchronous write to the clinical DB	API validates and applies before responding	Simple; 200 means queryable	Accept latency couples to DB health; retries duplicate without identity	Small volume with one trusted source
Log-first with async apply	Raw tier plus replicated log, workers apply	Acceptance stays up during apply outages; replay is native	Freshness is asynchronous; two durable writes	Baseline for millions of records per day
Broker as system of record	Long log retention replaces the raw tier	One durable store	Retention cost and no per-object policy; regulators expect immutable objects	Never for healthcare retention horizons
Per-record HTTP ingestion	One request per record	Fine-grained results	Request overhead at 3,000/s; batch atomicity is lost	Interactive sources with low volume
Managed integration engine	Off-the-shelf HL7/FHIR interface engine in front	Terminology and format handling	Opaque identity and retry semantics	Format mapping is the dominant problem

- IC4 stretch — strict per-patient ordering across facilities
 - Requirement becomes: one patient's records from two facilities apply in source-time order.
 - Partition key becomes (tenant, patient_token) after tokenization, which moves tokenization ahead of the log append.
 - Cost: the tokenization service joins the acceptance critical path and needs its own static-stability story.
- IC4 stretch — surviving failure set
 - Define the failure set for acknowledged records explicitly: local process crash, storage-node loss, and regional failure if required.
 - Regional survival adds cross-region raw replication before 202, which roughly doubles accept latency.
 - Tie replay retention to measured outage and catch-up time instead of a fixed 7 days.
- IC4 stretch — 10× load
 - 500 M records per day is about 30,000 records/s at peak and 2 TB/day raw.
 - Shard the log and apply fleet per tenant cell; move result rows to a partitioned, time-ordered store.
 - Bound replay so recovery from a 24 h outage still completes within a published window.
- IC4 stretch — cost and blast radius
 - The raw tier dominates cost at 100–170 TB compressed; tier it to cold storage after 90 days with restore time in the SLO.
 - Cell the pipeline per region and per tenant group so a poisoned partition or a bad transform release strands one cell.
 - Keep the reconciler independent of the apply fleet so it can prove loss during an incident.
- IC4 stretch — migration
 - A new clinical store is filled by replaying the raw tier under a new transform version.
 - Dual-write is unnecessary because every projection is rebuildable; cut over reads after reconciliation matches.
- Weakest assumption
 - Sources supply stable source_record_id and monotonic source_version.
 - A source that re-mints ids on retry defeats every deduplication rule downstream.

- Mitigation: fingerprint records by (subject_token, observed_at, code, value) at the validator and flag suspected duplicates for review rather than auto-merging.
- Over-engineering warning
 - Do not build a streaming join or real-time clinical alerting into the ingestion path.
 - Do not promise exactly-once across the source system; promise idempotent application under stable identity.
 - Do not adopt the metrics pipeline's downsampling; it is not a substitute for record retention.

Level signal — IC3 baseline vs IC4 stretch

Phase	IC3 must land	IC4 adds	Separating signal
Requirements	Record identity, per-patient ordering scope, acceptance meaning, retention horizon	Names partial-batch policy and deletion handling as explicit contracts	Asks what "accepted" means before drawing anything
Estimates	Daily volume, raw-tier size, partition count	Catch-up time from a 24 h outage as a published number	A number changes the replay capacity decision
Architecture	Control/data plane split, raw tier plus log, async apply with one trace	States what stays up during a control-plane outage	Every component has a mechanism and a price
Deep dive	Identity and version, 202 boundary, idempotent apply in one transaction	Crash between raw commit and log append; replay with a new transform version	Reasons across the object, log, and SQL boundary
Operations	Failure table, reconciliation, redaction	Replay capacity share, cell blast radius, rollback of a bad transform	Volunteers reconciliation gaps before being asked
Wrap	Recap and tradeoff	Names unstable source identity as the flaw first	Identifies the flaw before the interviewer does

Interview pushback and validation

- Q1: The source received 202 for the batch; why are records not queryable?
 - **IC3 baseline**
 - 202 means durably accepted, not applied; run state exposes applied and failed counts.
 - **IC4 stretch**
 - Publish a freshness SLO and a run watermark so the source can wait on a specific record identity.
- Q2: Just replay the dead-letter queue and move on.
 - **IC3 baseline**
 - Replay must reuse the original record identities or it duplicates observations.
 - **IC4 stretch**
 - Replays are child runs with lineage, a transform version, and an audit reason; the DLQ is a view over result rows, not a separate identity space.
- Q3: A correction arrives before the original record.
 - **IC3 baseline**
 - Both versions are stored; "current" is the highest source version at read time, so order of arrival does not matter.
 - **IC4 stretch**
 - Preserve clinical event time separately so a later-arriving earlier event does not appear as a reversal.
- Q4: The totals match, so the run is complete.
 - **IC3 baseline**
 - A duplicate and a missing record cancel out in a total; reconcile by identity.
 - **IC4 stretch**
 - Keep the reconciler independent of the apply fleet so it can prove loss during an apply incident.
- Q5: Why not guarantee global ordering across all facilities?

- **IC3 baseline**
 - No clinical requirement needs it, and it would serialize every facility behind one key.
- **IC4 stretch**
 - If cross-facility per-patient order is required, tokenize before the log and key by patient, accepting the tokenization dependency on the accept path.
- Q6: Error messages include the failing value so operators can fix it faster.
 - **IC3 baseline**
 - Error rows carry a code and a field path; the value is in the raw tier behind authorization.
 - **IC4 stretch**
 - Treat redaction failures as a security incident and alarm on them separately from validation failures.
- Q7 (premise reversal): Why a log and a raw tier at all? Write straight to the clinical database and return 200.
 - **IC3 baseline**
 - Acceptance would couple to database health, and a retry after a timeout would duplicate without a stable identity check on every write.
 - **IC4 stretch**
 - For one trusted source at low volume that is acceptable with the same unique keys; at millions of records with replays and regulators, the immutable raw tier is the requirement, and the log is what keeps acceptance up during apply outages.
- Q8: One malformed record in a batch of 10,000; fail the batch or continue?
 - **IC3 baseline**
 - Quarantine the record, continue the rest, and mark the run NEEDS_ATTENTION unless the source declared the batch atomic.
 - **IC4 stretch**
 - Make atomicity a source-level contract and enforce it with a run-level gate before any record becomes queryable.
- Validation scenarios
 - Source retries the same batch id with the same digest.
 - Source retries the same batch id with a different digest.
 - Crash after raw object commit and before log append.
 - Crash after log append and before the run row.
 - Consumer crash between the clinical upsert and the offset commit.
 - Correction with a higher version arriving before the original.
 - Replay of a whole run under a new transform version.
 - Skewed facility floods one partition during a replay.
 - Deletion request tracked through raw, normalized, and index copies.

Recall card

- "Accepted means raw object plus replicated log, never a transport ack."
- "Identity is source, record, and version; retries reuse it and corrections increment it."
- "Apply is insert-if-absent plus a result row in one transaction; offsets commit after."
- "Quarantine keeps identity so replay cannot double and reconciliation can explain every gap."
- "Totals that match prove nothing; reconcile by identity."
- Wrap-up script
 - "The design is a control plane for sources, runs, and replays over a data plane of an immutable raw tier, a keyed replicated log, and idempotent apply workers."
 - "The key invariant is that every record is identified by source, record id, and version, so a retry cannot double it and a correction cannot overwrite history."
 - "The main tradeoff is asynchronous application, which keeps acceptance up during store outages but makes freshness a separate SLO the source must read from run state."

- “My weakest assumption is that sources supply stable identities; I would validate that with fingerprint-based duplicate detection before trusting any source's retry behavior.”

References

- [Ranked question and report provenance](#).
- [OCI role framing](#).
- [Detailed API and schema companion](#).
- [Local first-hand report collection](#).
- [Metrics pipeline sibling](#): shared durable ingestion, offsets, and backpressure mechanisms.
- Reference check: 2026-09-10.
 - Capacity values, retention horizon, and architecture choices are proposed interview assumptions.
 - No claim of FHIR or HL7 endpoint semantics is made.
- [OCI Streaming overview](#): partitioned, replayable message log with bounded retention.
- [OCI Object Storage overview](#): immutable, encrypted, policy-retained raw tier.
- [OCI work requests](#): 202 plus operation id for asynchronous runs and replays.